ASTRAL

Technical Architecture & Design Document

v1.1 — Corrected | Confidential

> Supersedes v1.0. Five architectural corrections: HTTPS (was HTTP), python-oracledb (was cx_Oracle), Cookie-Harvest scraping (was plain httpx), Day-One rate limiting, Per-chapter checkpointing (was single integer).

---

0. Corrections from v1.0

❌→✅ Oracle Driver — cx_Oracle → python-oracledb (thin mode)

Was	`cx_Oracle` specified. Requires Oracle Instant Client binaries. No standard ARM64 Linux distribution. Docker build fails entirely on OCI Ampere A1 ARM VM.
Now	`python-oracledb` in thin mode. Pure Python, zero external binaries, ARM64 native via PyPI wheel. SQLAlchemy dialect: `oracle+oracledb://` . mTLS for Oracle Autonomous DB uses `ewallet.pem` — NOT `cwallet.sso` (thick-mode only).
Reason	`cx_Oracle` is formally deprecated by Oracle. `python-oracledb` is the official successor and runs on any architecture Python supports.

❌→✅ Transport Security — HTTP → HTTPS (Let's Encrypt + DuckDNS)

Was	HTTP-only deferred to post-v1. `NSAllowsArbitraryLoads` ATS exception in `Info.plist` . Exposes all API traffic to packet sniffing on any network.
Now	HTTPS from day one. Free DuckDNS subdomain (e.g. `astral-reader.duckdns.org`) → OCI public IP. Certbot container issues Let's Encrypt cert and auto-renews. No ATS exception needed — standard LE chain trusted by iOS.
Reason	This is a private app but travels over real networks. `NSAllowsArbitraryLoads` disables ATS globally, not just for one host. HTTPS costs nothing with LE + DuckDNS.

❌→✅ Scraping — plain httpx from datacenter IP → Cookie-Harvest + Playwright fallback

Was	`httpx` scraped directly from OCI datacenter IP. Datacenter IPs are blacklisted immediately by Cloudflare and manga aggregators. Rate limiting deferred to post-v1.
Now	**Primary:** iOS WKWebView (real Safari) passes Cloudflare natively. App extracts `cf_clearance` cookies + user-agent after page load and sends them with the scrape request. Backend `httpx` reuses those cookies to impersonate the Safari session. Playwright + stealth is the fallback for sites needing per-page JS rendering. Rate limiting with exponential backoff is a day-one `BaseScraper` requirement.
Reason	The iOS browser already solves Cloudflare. Re-using its cookies avoids a headless browser server-side in most cases. Playwright ARM64 is officially supported by Microsoft on Ubuntu 22+.

❌→✅ Checkpointing — `last_scraped_chapter_index` INTEGER removed

Was	Single integer `last_scraped_chapter_index` on `scrape_jobs`. If chapter 10 fails but 11–15 succeed, retry from index 10 re-scrapes 11–15. Non-linear failures silently mishandled.
Now	Column **REMOVED**. Per-chapter `scrape_status` (`pending` | `scraped` | `failed`) on `comic_chapters` / `fanfic_chapters` IS the checkpoint. Retry queries `WHERE scrape_status IN ('failed', 'pending')`. Non-linear failures handled correctly. Already-scraped chapters never re-scraped.
Reason	Per-chapter status was already in the schema. The integer was redundant and incorrect for non-linear failure patterns.

❌→✅ Rate Limiting — deferred post-v1 → day-one BaseScraper requirement

Was	Rate limiting and exponential backoff listed as post-v1 deferred item. AO3 enforces strict documented request limits.
Now	`BaseScraper._fetch()` enforces: per-source configurable minimum request delay, exponential backoff with jitter on 429/503, `Retry-After` header respected. AO3 default: 2s between requests, max 3 retries. These are constructor arguments overridable per scraper subclass.

Reason	AO3 will actively throttle or ban scrapers that ignore rate limits. This cannot be deferred — the scraper will not function reliably without it.

---

1. Project Overview

1.1 Summary

Astral is a private, single-user iOS reading application for manga/manhwa comics and fan fiction. It is not intended for App Store distribution and is sideloaded to a personal iPhone via Xcode with a free Apple developer account. The backend runs on Oracle Cloud Infrastructure (OCI) Free Tier using a single Ampere A1 ARM VM (4 cores, 24GB RAM).

> **Single-User Trust Model:** The iOS app is the sole client. No authentication layer in v1. The backend is implicitly trusted — accessible only from the personal iPhone and local development machine. This is an intentional trade-off for a private sideloaded application.

1.2 Architecture

Component	Stack
iOS App	SwiftUI, iOS 17.2+, SwiftData (local persistence), URLSession / APIClient (async/await)
Backend API	Python 3.11+, FastAPI, Uvicorn, Nginx reverse proxy
Task Queue	ARQ + Redis — async scrape jobs, status tracking, nightly cleanup cron
DB Driver	python-oracledb thin mode — pure Python, ARM64 native, no Instant Client required
Database	Oracle Autonomous Database Free Tier (1 OCPU, 20GB). mTLS via `ewallet.pem` from wallet ZIP
File Storage	OCI Block Volume (200GB) served by Nginx as `/static/` for scraped images
TLS	Let's Encrypt cert on DuckDNS subdomain. Certbot container auto-renews. Standard iOS trust.
Scraping	iOS WKWebView harvests CF cookies → backend httpx reuses them. Playwright ARM64 fallback.
Deployment	Single OCI Ampere A1 VM. Docker Compose split: api stack + worker stack

1.3 Technology Decisions

Decision	Rationale

python-oracledb thin vs cx_Oracle	`cx_Oracle` is deprecated and requires Oracle Instant Client binaries unavailable for ARM64 Linux. `python-oracledb` thin mode is pure Python, installs via pip, runs on any architecture, and is Oracle's official successor.
mTLS wallet format (thin mode)	Oracle Autonomous DB requires mTLS. Thin mode uses `ewallet.pem` from the wallet ZIP. NOT `cwallet.sso` — that format is thick-mode only. The PEM file is mounted into the Docker container via a secrets volume.
HTTPS via DuckDNS + Let's Encrypt	Free subdomain points to OCI public IP. Let's Encrypt issues a valid cert. Certbot in Docker renews automatically. iOS trusts the standard LE chain — no ATS exceptions.
Cookie-Harvest scraping	iOS WKWebView is real Safari — passes Cloudflare natively. App extracts `cf_clearance` cookies and user-agent after page load. Sent to backend with scrape request. Backend httpx reuses them, impersonating the Safari session.
Playwright as fallback	For sites requiring per-page JS rendering. `playwright-stealth` patches fingerprint vectors. Chromium ARM64 supported officially on Ubuntu 22+. Shared pool of max 2 browser instances.
ARQ over Celery	ARQ is built on asyncio. Scraping is I/O-bound. Celery requires sync/async bridging and heavier broker setup. ARQ uses Redis only, integrates cleanly with FastAPI, and tracks job IDs natively.
Block Volume vs Object Storage	OCI Object Storage free tier caps at 20GB. Block volume gives 200GB. Manga reading is sequential small-file reads — block volume + Nginx is the right approach.
SwiftData	iOS 17.2+ target. Native SwiftUI integration. Used for local state only: reading progress, download status, preferences. Server DB is always source of truth. Minimum 17.2 avoids known predicate bugs.

---

2. Repository & Project Structure

2.1 Monorepo Layout

```
astral/
├── .gitignore                    # covers ios/ and backend/ both
├── README.md
├── docs/
│   └── api-contracts/
│       ├── comics.yaml
```

```
│       ├── fanfic.yaml
│       ├── scrape.yaml
│       └── shared.yaml
├── ios/
│   └── Astral/                    # Xcode project root
└── backend/
    ├── docker-compose.yml         # fastapi + nginx + certbot
    ├── docker-compose.worker.yml # arq_worker + redis
    ├── .env.local                 # local dev template (gittracked)
    ├── .env.oci                   # OCI production (gitignored)
    └── app/
```

2.2 Backend Directory

```
backend/app/
├── main.py                        # FastAPI app init, router registration
├── dependencies.py                # Shared FastAPI Depends functions
├── worker.py                      # ARQ WorkerSettings, cron job registration
│
├── core/
│   ├── config.py                  # Pydantic BaseSettings — reads .env
│   └── constants.py               # Enums, source keys, rate limit configs
│
├── api/v1/routes/
│   ├── comics.py                  # /api/v1/comics/*
│   ├── fanfic.py                  # /api/v1/fanfic/*
│   ├── scrape.py                  # /api/v1/scrape/*
│   ├── authors.py                 # /api/v1/authors/*
│   └── health.py                  # /api/v1/health
│
├── models/
│   ├── comic.py                   # ORM: Comics, Tags, Chapters, Pages
│   ├── fanfic.py                  # ORM: Fanfics, FanficChapters
│   ├── author.py                  # ORM: Authors, join tables
│   ├── scrape.py                  # ORM: ScrapeJobs
│   └── progress.py                # ORM: ReadingProgress
│
├── schemas/
│   ├── comic.py / fanfic.py / scrape.py / shared.py
│
├── services/
│   ├── comic_service.py / fanfic_service.py
│   ├── scrape_service.py / progress_service.py
│
├── scrapers/
│   ├── base.py                    # BaseScraper — rate limiting, backoff, cookie reuse
│   ├── comic/
│   │   ├── nhentai.py             # NhentaiScraper(BaseScraper)
│   │   ├── toongod.py             # ToongodScraper(BaseScraper)
```

```
│   │   └── hentai20.py           # Hentai20Scraper(BaseScraper)
│   └── fanfic/
│       ├── ao3.py                # AO3Scraper — stricter rate limits
│       └── ffnet.py              # FanfictionNetScraper(BaseScraper)
│
├── tasks/
│   ├── comic_scrape_task.py      # ARQ task: per-chapter scrape loop
│   ├── fanfic_scrape_task.py     # ARQ task: per-chapter scrape loop
│   └── cleanup_task.py           # Nightly cron: hard-delete rows > 5d old
│
├── db/
│   └── database.py               # python-oracledb engine, session factory
│
└── utils/
    ├── file_storage.py           # Block volume path helpers
    ├── http_client.py            # Shared httpx AsyncClient singleton
    ├── playwright_client.py      # Playwright browser pool (fallback scraper)
    └── image_utils.py            # Thumbnail generation, image validation
```

2.3 iOS Project Structure

```
ios/Astral/
├── Astral.xcodeproj
├── AstralApp.swift               # @main, ModelContainer setup
├── Info.plist                    # No NSAllowsArbitraryLoads — HTTPS only
│
├── App/
│   ├── AppState.swift            # activeTab, sidebar state
│   ├── RootView.swift            # Tab switcher shell
│   └── TabBarView.swift          # Custom bottom tab bar (not TabView)
│
├── Packages/
│   ├── Core/Sources/Core/
│   │   ├── Models/               # SwiftData @Model classes
│   │   └── Utilities/
│   ├── Networking/Sources/Networking/
│   │   ├── APIClient.swift       # async/await URLSession wrapper
│   │   ├── Endpoints.swift       # All endpoint definitions
│   │   ├── CookieStore.swift     # Persists harvested CF cookies
│   │   └── DTOs/                 # Codable response models
│   ├── DesignSystem/Sources/DesignSystem/
│   │   ├── Colors.swift          # Token-based gold/dark palette
│   │   ├── Typography.swift      # Inter font scale
│   │   ├── Components/           # Cards, buttons, progress bar, badges
│   │   └── Modifiers/            # Reusable SwiftUI view modifiers
│   ├── ComicFeature/Sources/ComicFeature/
│   │   ├── Library/              # Grid view, comic cards
│   │   ├── Reader/               # Image viewer, page/chapter controls
│   │   ├── Browser/              # WKWebView + cookie extraction + scrape
```

```
│   │   ├── Scrapes/              # Job list: in-progress, partial, done
│   │   └── ViewModels/
│   └── FanficFeature/Sources/FanficFeature/
│       ├── Library/              # Full-width list, AO3-style filter
│       ├── Reader/               # Text reader, scroll tracking
│       ├── Browser/
│       ├── Scrapes/
│       └── ViewModels/
└── Resources/
    ├── Assets.xcassets
    └── Fonts/                    # Inter Variable font files
```

2.4 Docker Compose Services

Service	Role
`docker-compose.yml`	Services: `fastapi` (Uvicorn on :8000), `nginx` (:443 + :80→redirect), `certbot` (auto-renew). Network: `astral_net`. Block volume and wallet mounted as volumes.
`docker-compose.worker.yml`	Services: `redis` (alpine, persistent volume on block vol), `arq_worker`. Joins `astral_net`. Same `.env` file. Started independently when scraping is needed.
Nginx role	SSL termination with Let's Encrypt cert. Proxies `/api/` → `fastapi:8000`. Serves `/static/` → `/mnt/astral-media/`. Port 80 → 301 HTTPS.
Certbot role	Sidecar container. Issues cert for DuckDNS domain on first boot. Renewal checked every 12h — LE renews when <30d remaining. Cert in named volume shared with Nginx.
Oracle Wallet	`ewallet.pem` mounted read-only into `fastapi` container from host. Never baked into Docker image. Path set via `ORACLE_WALLET_PATH` env var.

---

3. Scraper Architecture

> **Core Principle:** The iOS WKWebView IS the Cloudflare solver. It runs real Safari — no datacenter heuristics, no bot fingerprinting. After the user navigates to a story page, the app extracts cookies and user-agent. The backend reuses these credentials for all subsequent requests, impersonating the Safari session. Playwright is a fallback only — used when per-chapter pages require fresh JS rendering beyond what cookies can unlock.

3.1 Cookie-Harvest Flow (Primary Path)

Step	Description

1. User browses	Opens story in the in-app browser (WKWebView). Real Safari passes Cloudflare JS challenge naturally.
2. Page load complete	WKWebView `navigationDelegate` fires `webView(_:didFinish:)`. App calls `WKWebsiteDataStore.httpCookieStore.allCookies()`.
3. Cookie extraction	App filters cookies for source domain. Captures `cf_clearance`, session cookies, and current user-agent from `WKWebView.evaluateJavaScript("navigator.userAgent")`.
4. Scrape request	`POST /api/v1/scrape/comic` with: `{ url, source_key, cookies: [{name, value, domain}], user_agent }`
5. Backend stores	Cookies written to Redis. Key: `"cookies:{source_key}"`, TTL: `COOKIE_CACHE_TTL_SECS` (default 24h).
6. httpx requests	Backend constructs httpx client with `headers["User-Agent"] = user_agent` and cookies from Redis. Requests appear to CF as the same Safari session.
7. Cookie refresh	If backend receives 403/503 from source: returns HTTP 428 to iOS. App shows "Browser refresh needed" prompt. User re-navigates — new cookies auto-harvested on page load.

3.2 Playwright Fallback

Used when sites rotate session tokens per-chapter-page or embed signed image URLs requiring full browser context. Triggered when: (a) scraper class sets `requires_browser = True`, or (b) backend receives >3 consecutive 403s on a cookie-harvest session.

ARM64 support	Official Microsoft `playwright` Python image supports ARM64 on Ubuntu 22. Use `ubuntu-22.04` tagged variant explicitly. `playwright install chromium` downloads the ARM64 binary.
Browser pool	`playwright_client.py` manages a shared pool of max 2 concurrent browser instances (~300MB RAM each; A1 VM has 24GB). ARQ tasks acquire a slot via `asyncio.Semaphore`.
Cookie re-use	After Playwright solves a challenge, extracted cookies go to Redis with `source_key` key. Subsequent requests use httpx — browser slot released.
playwright-stealth	Patches `navigator.webdriver`, plugin fingerprints, canvas fingerprints, and other bot-detection vectors in Chromium's headless mode.

3.3 BaseScraper Contract

```
class BaseScraper(ABC):
    source_key: str                      # "nhentai" | "ao3" | etc.
    content_type: str                    # "comic" | "fanfic"
    requires_browser: bool = False       # True = Playwright fallback
    request_delay_seconds: float = 1.0   # Min delay between requests
    max_retries: int = 3

    # Must implement:
    @abstractmethod
    async def get_story_metadata(self, url: str) -> StoryMetadata: ...
    @abstractmethod
    async def get_chapter_list(self, story_url: str) -> list[ChapterInfo]: ...
    @abstractmethod
    async def get_chapter_pages(self, chapter_url: str) -> list[PageInfo]: ...
    @abstractmethod
    async def get_chapter_text(self, chapter_url: str) -> str: ...

    # Provided by BaseScraper — do not override:
    async def _fetch(self, url: str) -> str:
        """
        1. Load cookies for source_key from Redis.
        2. asyncio.sleep(request_delay_seconds) — rate limiting.
        3. GET with retry loop:
           - 429 / 503: respect Retry-After header; backoff = 2^attempt + jitter
           - 403: raise CookieExpiredError → caller returns 428 to iOS
           - 200: return response text
        4. After max_retries exhausted: raise ScraperError.
        """

    async def download_image(self, url: str, dest_path: str) -> str: ...
    # Uses _fetch() — inherits rate limiting and backoff automatically
```

3.4 Per-Source Configuration

Source	requires_browser	request_delay	Notes
nhentai	False (primary)	1.5s	CF protected. Cookie-harvest usually sufficient.
toongod	False (primary)	1.0s	CF protected. Monitor for per-page token rotation.
hentai20	True (Playwright)	2.0s	Requires browser for chapter image URL extraction.
ao3	False — no CF	2.0s	No Cloudflare. Strict rate limits. Retry-After required.
ffnet	False (primary)	1.5s	CF protected. Cookie-harvest.

3.5 Scrape Job Flow

Step	Description
1. iOS trigger	`POST /api/v1/scrape/{comic|fanfic}` : `{ url, source_key, cookies[], user_agent }`
2. Cookie store	Cookies written to Redis. `ScrapeJob` row inserted: `status = queued` .
3. ARQ enqueue	Task pushed with `job_id` . Returns `{ job_id }` to iOS immediately.
4. Metadata fetch	Scraper fetches story metadata and chapter list. `total_chapters` updated on story row.
5. Chapter loop	For each chapter `WHERE scrape_status = pending` : fetch → parse → persist → `scrape_status = scraped` .
6. Partial failure	Exception on a chapter: `chapter.scrape_status = failed` . Job continues to next chapter. Job `status = partial` on completion.
7. Partial read	112 scraped chapters fully readable while 38 remain failed. App shows partial indicator on library card.
8. iOS poll	`GET /api/v1/scrape/{job_id}` → `{ status, chapters_scraped, chapters_failed, total_chapters }`
9. Retry	`POST /api/v1/scrape/{job_id}/retry` . Worker queries `WHERE scrape_status IN ('failed', 'pending')` . Already-scraped chapters skipped entirely.
10. Delta update	`POST /api/v1/scrape/{story_id}/update` . Fetches chapter list from source. Inserts only chapters with `chapter_number > max(existing)` . Scrapes new chapters only.

---

4. Database Schema

Oracle Autonomous Database Free Tier. Driver: **python-oracledb thin mode.** SQLAlchemy dialect: `oracle+oracledb://` . mTLS via `ewallet.pem` (NOT `cwallet.sso`). No Alembic in v1 — schema managed by `create_all()` on startup. All tables include `deleted_at TIMESTAMP NULL` for soft delete. A nightly ARQ cron task hard-deletes rows where `deleted_at < NOW() - 5 days` .

> **Checkpointing Note:** `last_scraped_chapter_index` INTEGER from v1.0 is **REMOVED**. The per-chapter `scrape_status` column on `comic_chapters` and `fanfic_chapters` IS the checkpoint. Retry = `SELECT * FROM chapters WHERE scrape_status IN ('failed', 'pending')` . Non-linear

> failures handled correctly.

4.1 authors

Column	Type	Constraints	Notes
id	UUID	PK, gen_random_uuid()	
name	VARCHAR(500)	NOT NULL, UNIQUE INDEX	Exact-match dedup in v1
created_at	TIMESTAMP	NOT NULL, default NOW()	
deleted_at	TIMESTAMP	NULL	Soft delete

4.2 comics

Column	Type	Constraints	Notes
id	UUID	PK	
title	VARCHAR(1000)	NOT NULL	
source_key	VARCHAR(50)	NOT NULL	nhentai | toongod | hentai20
source_url	VARCHAR(2000)	NOT NULL, UNIQUE	Canonical scrape URL
source_id	VARCHAR(200)	NULL	Source-native ID if extractable
thumbnail_path	VARCHAR(1000)	NULL	Block vol relative path; NULL = blanked by user
description	TEXT	NULL	
total_chapters	INTEGER	NOT NULL, default 0	Updated per scrape
total_pages	INTEGER	NOT NULL, default 0	Sum across all chapters
language	VARCHAR(50)	NULL	
status	VARCHAR(20)	NOT NULL, default pending	pending | partial | complete
scrape_job_id	UUID	FK scrape_jobs.id	Latest job reference
created_at	TIMESTAMP	NOT NULL	
updated_at	TIMESTAMP	NOT NULL	
deleted_at	TIMESTAMP	NULL	Soft delete

4.3 comic_authors (join table)

Column	Type	Constraints	Notes
comic_id	UUID	FK comics.id, NOT NULL	
author_id	UUID	FK authors.id, NOT NULL	
role	VARCHAR(50)	NULL	artist | writer | circle | group
(PK)	COMPOSITE	(comic_id, author_id, role)	

4.4 tags (comic-only)

Column	Type	Constraints	Notes
id	UUID	PK	
name	VARCHAR(200)	NOT NULL	Lowercased, trimmed on insert
tag_type	VARCHAR(50)	NOT NULL	genre | character | parody | artist | group | language | category
(UNIQUE)	INDEX	(name, tag_type)	Dedup by name + type
created_at	TIMESTAMP	NOT NULL	

4.5 comic_tags (join table)

Column	Type	Constraints	Notes
comic_id	UUID	FK comics.id, NOT NULL	
tag_id	UUID	FK tags.id, NOT NULL	
(PK)	COMPOSITE	(comic_id, tag_id)	

4.6 comic_chapters

Column	Type	Constraints	Notes
id	UUID	PK	
comic_id	UUID	FK comics.id, NOT NULL	

chapter_number	FLOAT	NOT NULL	Float supports ch 12.5 specials
title	VARCHAR(500)	NULL	
source_url	VARCHAR(2000)	NULL	
total_pages	INTEGER	NOT NULL, default 0	
scrape_status	VARCHAR(20)	NOT NULL	**pending | scraped | failed ← CHECKPOINT COLUMN**
created_at	TIMESTAMP	NOT NULL	
deleted_at	TIMESTAMP	NULL	
(UNIQUE)	INDEX	(comic_id, chapter_number)	

4.7 pages

`file_path` is relative to block volume mount root. Nginx serves: `https://{host}/static/{file_path}`. iOS builds full URL from `STATIC_BASE_URL` env var.

Column	Type	Constraints	Notes
id	UUID	PK	
chapter_id	UUID	FK comic_chapters.id, NOT NULL	
page_number	INTEGER	NOT NULL	1-indexed display order
file_path	VARCHAR(1000)	NOT NULL	Relative to block volume root
source_url	VARCHAR(2000)	NULL	Original image URL from source
width_px	INTEGER	NULL	Stored at scrape time
height_px	INTEGER	NULL	
(UNIQUE)	INDEX	(chapter_id, page_number)	

4.8 fanfics

Column	Type	Constraints	Notes
id	UUID	PK	

title	VARCHAR(1000)	NOT NULL	
source_key	VARCHAR(50)	NOT NULL	ao3 | ffnet
source_url	VARCHAR(2000)	NOT NULL, UNIQUE	
source_id	VARCHAR(200)	NULL	AO3 work ID / FFNet story ID
summary	TEXT	NULL	
fandom	VARCHAR(500)	NULL	
relationship	VARCHAR(500)	NULL	Pairing / ship
characters	VARCHAR(1000)	NULL	Comma-sep or JSON array
rating	VARCHAR(20)	NULL	G | T | M | E
warnings	VARCHAR(500)	NULL	Archive warnings string
completion_status	VARCHAR(20)	NOT NULL	complete | ongoing | abandoned
word_count	INTEGER	NULL	Total across all chapters
total_chapters	INTEGER	NOT NULL, default 0	
published_at	TIMESTAMP	NULL	Original publish date on source
updated_at_source	TIMESTAMP	NULL	Last updated on source site
language	VARCHAR(50)	NULL	
thumbnail_path	VARCHAR(1000)	NULL	User-set cover; nullable
scrape_job_id	UUID	FK scrape_jobs.id	
created_at	TIMESTAMP	NOT NULL	
updated_at	TIMESTAMP	NOT NULL	
deleted_at	TIMESTAMP	NULL	Soft delete

4.9 fanfic_authors (join table)

Column	Type	Constraints	Notes
fanfic_id	UUID	FK fanfics.id, NOT NULL	
author_id	UUID	FK authors.id, NOT NULL	
(PK)	COMPOSITE	(fanfic_id, author_id)	

4.10 fanfic_chapters

Column	Type	Constraints	Notes
id	UUID	PK	
fanfic_id	UUID	FK fanfics.id, NOT NULL	
chapter_number	INTEGER	NOT NULL	
title	VARCHAR(500)	NULL	
content	TEXT	NOT NULL	Full chapter HTML/text stored in DB
word_count	INTEGER	NULL	Per-chapter word count
source_url	VARCHAR(2000)	NULL	
scrape_status	VARCHAR(20)	NOT NULL	**pending | scraped | failed ← CHECKPOINT COLUMN**
created_at	TIMESTAMP	NOT NULL	
deleted_at	TIMESTAMP	NULL	
(UNIQUE)	INDEX	(fanfic_id, chapter_number)	

4.11 scrape_jobs (`last_scraped_chapter_index` REMOVED)

Column	Type	Constraints	Notes
id	UUID	PK	
content_type	VARCHAR(10)	NOT NULL	comic | fanfic
story_id	UUID	NOT NULL	FK to comics.id OR fanfics.id (polymorphic)
source_url	VARCHAR(2000)	NOT NULL	URL that was scraped
source_key	VARCHAR(50)	NOT NULL	Scraper identifier
job_type	VARCHAR(20)	NOT NULL	initial | delta | retry
status	VARCHAR(20)	NOT NULL	queued | running | partial | complete | failed

total_chapters	INTEGER	NULL	Discovered at scrape start
chapters_scraped	INTEGER	NOT NULL, default 0	Incremented per successful chapter
chapters_failed	INTEGER	NOT NULL, default 0	Incremented per failed chapter
error_message	TEXT	NULL	Last failure reason
started_at	TIMESTAMP	NULL	
completed_at	TIMESTAMP	NULL	
created_at	TIMESTAMP	NOT NULL	
deleted_at	TIMESTAMP	NULL	5-day soft delete
~~last_scraped_chapter_index~~	~~REMOVED~~	~~REMOVED~~	Per-chapter scrape_status is the checkpoint

4.12 reading_progress

One row per story. Upserted on every significant navigation event. Progress bar = `last_chapter_number / total_chapters`. `scroll_offset_percent` is fanfic-only: 0.0–1.0 through the current chapter text.

Column	Type	Constraints	Notes
id	UUID	PK	
content_type	VARCHAR(10)	NOT NULL	comic | fanfic
story_id	UUID	NOT NULL	FK to comics.id OR fanfics.id
last_chapter_id	UUID	NULL	FK to chapter table
last_chapter_number	INTEGER	NOT NULL, default 1	Progress bar numerator
last_page_number	INTEGER	NULL	Comic only — page within chapter
scroll_offset_percent	FLOAT	NULL	Fanfic only — 0.0 to 1.0
updated_at	TIMESTAMP	NOT NULL	Updated on each read event
(UNIQUE)	INDEX	(content_type, story_id)	One row per story

5. iOS Data Layer

5.1 Cookie Extraction (WKWebView)

```
// BrowserViewModel.swift
func extractCookiesAndScrape(url: URL, sourceKey: String) async {
    let store = webView.configuration.websiteDataStore.httpCookieStore
    let cookies = await store.allCookies()
    let sourceCookies = cookies
        .filter { $0.domain.contains(sourceKey) }
        .map { CookieDTO(name: $0.name, value: $0.value, domain: $0.domain) }

    let ua = try? await webView.evaluateJavaScript("navigator.userAgent") as? String

    let request = ScrapeRequest(
        url: url.absoluteString,
        sourceKey: sourceKey,
        cookies: sourceCookies,
        userAgent: ua ?? "Mozilla/5.0 (iPhone; CPU iPhone OS 17_2)"
    )
    let job = try await apiClient.request(.initiateScrape(request))
    // Store job.id, begin polling
}
```

5.2 SwiftData Models (On-Device)

SwiftData mirrors API response DTOs — not the server DB schema. Used for local state only. Server is source of truth.

Model	Fields
LocalComic	id, title, sourceKey, thumbnailPath, totalChapters, status, isDownloaded, lastReadChapterNumber, progressPercent, addedAt
LocalComicChapter	id, comicId, chapterNumber, title, totalPages, isDownloaded, scrapeStatus
LocalFanfic	id, title, sourceKey, summary, fandom, rating, completionStatus, wordCount, totalChapters, isDownloaded, lastReadChapterNumber, scrollOffsetPercent, progressPercent, addedAt
LocalFanficChapter	id, fanficId, chapterNumber, title, wordCount, isDownloaded, localTextPath, scrapeStatus
LocalAuthor	id, name
UserPreferences	id (singleton), comicScrollDirection, fanficFontSize, fanficLineHeight, fanficBackground, readerBrightness

LocalScrapeJob	id, contentType, storyId, status, chaptersScraped, totalChapters, createdAt, completedAt

> **SwiftData Deployment Target:** Minimum iOS 17.2, NOT 17.0. Known bugs in 17.0–17.1: complex predicates on optional to-many relationships silently fail. All `@Query` predicates that filter on optionals must be tested on the 17.2 simulator specifically. Prefer in-memory filtering for small datasets over complex `@Query` predicates.

5.3 Offline Mode

State	Behaviour
Online	All content from API. SwiftData synced on each successful response.
Backend unreachable	Netflix-style: banner shown. Only `isDownloaded=true` stories and chapters visible. Undownloaded content shows "Unavailable offline" placeholder.
Cookie 428 response	App shows "Browser refresh needed" prompt. User re-navigates in browser — new cookies auto-harvested on page load.
Download limits	5GB per tab (Comic / Fanfic separately) in Documents directory. `DownloadService` blocks downloads when at limit. Storage usage shown in Settings.

---

6. UI Architecture

6.1 Navigation Shell

Component	Description
RootView	ZStack of ComicTabView and FanficTabView. Custom tab bar at bottom. Active tab by `AppState.activeTab`.
Tab bar	Custom SwiftUI — NOT `TabView`. Two pill items: Comic (book icon), Fanfic (scroll icon). Metallic gold active indicator.
Sidebar (both)	Custom slide-over drawer: `ZStack` + `.offset(x:)` + spring animation. Toggled by top-right icon. Never `NavigationSplitView` — it collapses to a stack on iPhone compact width.
Comic sidebar	Recents (last 5 dropdown), Library, Read, Browse, Scrapes
Fanfic sidebar	Recent Stories, Library. Top bar (fanfic only): Search, Downloads.

6.2 Comic Tab

View	Description
LibraryView	`LazyVGrid` , 2 columns. Card: thumbnail, title, author(s), progress bar (`lastChapterNumber / totalChapters * 100%`), downloaded badge, partial-scrape indicator.
ReaderView	Full-screen, black background. Tap center toggles HUD (page #, chapter title, prev/next). Pinch-to-zoom. Toggle: vertical scroll (webtoon) vs horizontal page-flip. Direction persisted globally.
BrowserView	WKWebView. Source picker segmented: nhentai | toongod | hentai20. Floating Scrape button on story pages. Triggers `extractCookiesAndScrape()` on tap.
ScrapesView	List grouped: In Progress (spinner, X/Y chapters), Partial (warning, retry button), Completed (checkmark), Failed (X, retry). Pull-to-refresh.

6.3 Fanfic Tab

View	Description
LibraryView	Full-width list. Row: thumbnail, title, author, summary excerpt, tag pills (fandom, rating, status, word count). AO3-style filter sheet. Sort: date added, word count, last updated, title.
FilterView	Bottom sheet. Fields: Fandom (text), Rating (multi-select), Completion Status, Word Count range slider, Language. Applied as query params to `GET /api/v1/fanfic` .
ReaderView	ScrollView of chapter text. Chapter picker at top. Reader bar: font size 14–22pt, line height 1.4–1.8x, background (dark/sepia/paper). Scroll offset tracked, debounced 5s, synced on exit.
BrowserView	WKWebView. Source picker: AO3 | FFNet. Same cookie-harvest + scrape flow as comic browser.

6.4 Design System

Token	Value
Palette	Background `#0A0A0B` , surface `#141416` , elevated `#1E1E22` , border `#2A2A30` , muted `#6B6B7A` , body `#C8C8D4` , white `#F0F0F8` , gold `#C9A84C`
Gold treatment	Active tab indicators and primary buttons use metallic gold with a subtle radial gradient shimmer (`CAGradientLayer`). Liquid-glass reflective effect via semi-transparent highlight overlay.

Typography	Inter Variable (custom font). Display 34pt bold. Title 22pt semibold. Body 16pt regular. Caption 12pt regular. Dynamic Type compatible.
Motion	Sidebar spring: 0.35s, damping 0.85. Tab crossfade: 0.2s. Reader HUD opacity: 0.15s ease. Scrape in-progress: continuous pulse animation.
Reader fonts	Fanfic: Inter, user-set 14–22pt, line height 1.4–1.8x. Backgrounds: `#0A0A0B` (dark), `#2C1810` (sepia), `#F5F0E8` (paper).

---

7. Key API Contracts

7.1 Endpoints

Method	Path	Description
GET	`/api/v1/health`	Health check
GET	`/api/v1/comics`	Library. Params: page, page_size, sort, filter
GET	`/api/v1/comics/{id}`	Comic detail + chapter list
GET	`/api/v1/comics/{id}/chapters/{cid}/pages`	Page list for chapter
PATCH	`/api/v1/comics/{id}`	Update thumbnail or title
DELETE	`/api/v1/comics/{id}`	Soft delete
GET	`/api/v1/fanfic`	Library. AO3-style filter params
GET	`/api/v1/fanfic/{id}`	Fanfic detail + chapter list
GET	`/api/v1/fanfic/{id}/chapters/{cid}`	Chapter text content
DELETE	`/api/v1/fanfic/{id}`	Soft delete
GET	`/api/v1/authors/{id}`	Author + all stories across both types
POST	`/api/v1/scrape/comic`	Initiate. Body: `{url, source_key, cookies[], user_agent}`
POST	`/api/v1/scrape/fanfic`	Initiate. Same body shape.
GET	`/api/v1/scrape/{job_id}`	Job status + chapter counts
POST	`/api/v1/scrape/{job_id}/retry`	Resume failed/pending chapters only

POST	`/api/v1/scrape/{story_id}/update`	Delta: new chapters only
GET	`/api/v1/scrape`	All jobs, paginated
PUT	`/api/v1/progress/comic/{story_id}`	Upsert: chapter, page
PUT	`/api/v1/progress/fanfic/{story_id}`	Upsert: chapter, scroll offset
GET	`/api/v1/progress/{type}/{story_id}`	Get story progress

7.2 Scrape Request Body

```
{
  "url": "https://nhentai.net/g/12345/",
  "source_key": "nhentai",
  "cookies": [
    { "name": "cf_clearance", "value": "abc123...", "domain": ".nhentai.net" }
  ],
  "user_agent": "Mozilla/5.0 (iPhone; CPU iPhone OS 17_2 like Mac OS X)..."
}
```

7.3 Pagination Envelope

```
{
  "items": [],
  "total": 142,
  "page": 1,
  "page_size": 20,
  "total_pages": 8,
  "has_next": true
}
```

---

8. Infrastructure & Deployment

8.1 HTTPS Setup

Component	Description
DuckDNS	Free subdomain (e.g. `astral-reader.duckdns.org`) → OCI reserved public IP. DuckDNS token in env var. Docker cron updates A record if IP changes (reserved IP is static — this is insurance).
Let’s	Certbot container issues cert for DuckDNS domain on first boot. Cert in named volume (`letsencrypt`) shared read-only with Nginx. Renewal checked every 12h —

Encrypt	LE renews when <30d remaining.
Nginx TLS	Listens on 443 with SSL cert. Port 80 → 301 HTTPS. Standard LE chain trusted by iOS with no ATS configuration required.
No ATS override	`Info.plist` contains no `NSAllowsArbitraryLoads`. No `NSExceptionDomains`. Clean HTTPS throughout. Replaces v1.0 plaintext HTTP + broad ATS exception.

8.2 OCI Resources

Resource	Allocation
VM	Ampere A1 ARM — 4 cores, 24GB RAM, 1 reserved public IP. All services run here.
Block Volume	200GB total. `/mnt/astral-media/` for scraped images. Nginx serves as `/static/`.
Oracle ADB	Free: 1 OCPU, 20GB. mTLS required. Wallet ZIP from OCI console. `ewallet.pem` extracted and mounted into fastapi container at `/app/wallet/ewallet.pem`.
Redis	Alpine container. Persistent volume on block volume. Task queue for ARQ and cookie cache (source cookies, 24h TTL).
Playwright	Chromium installed in `arq_worker` container via `playwright install chromium`. ARM64 binary. Shared browser pool: max 2 instances via `asyncio.Semaphore`.

8.3 Environment Variables

Variable	Description	Local example
`DATABASE_URL`	oracle+oracledb:// connection string	`oracle+oracledb://user:pass@host:1521/service`
`ORACLE_WALLET_PATH`	Path to `ewallet.pem` in container	`/app/wallet/ewallet.pem`
`REDIS_URL`	Redis connection URL	`redis://redis:6379/0`
`BLOCK_VOLUME_PATH`	Absolute path to media mount	`/mnt/astral-media`
`STATIC_BASE_URL`	HTTPS base URL for image serving	`https://astral-reader.duckdns.org/static`
`API_BASE_URL`	iOS app API root (in	`https://astral-reader.duckdns.org/api/v1`

	`AppConfig.swift`)	
`DUCKDNS_TOKEN`	DuckDNS API token	`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`
`DUCKDNS_DOMAIN`	DuckDNS subdomain prefix	`astral-reader`
`ARQ_MAX_JOBS`	Concurrent ARQ worker slots	`3`
`SOFT_DELETE_DAYS`	Days before hard delete	`5`
`COOKIE_CACHE_TTL_SECS`	Redis TTL for harvested cookies	`86400`

8.4 python-oracledb Connection

```
# db/database.py
import oracledb
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

# Thin mode is DEFAULT — no init_oracle_client() call needed
# Oracle Autonomous DB requires mTLS — pass ewallet.pem path
connect_args = {
    "config_dir":      str(settings.oracle_wallet_dir),
    "wallet_location": str(settings.oracle_wallet_dir),
    "wallet_password": settings.oracle_wallet_password,
}

# Dialect: oracle+oracledb  (NOT oracle+cx_oracle)
engine = create_engine(
    settings.database_url,
    connect_args=connect_args,
    pool_size=5,
    pool_timeout=30,
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

---

9. Engineering Conventions

9.1 Backend

- All routes prefixed `/api/v1/` . No exceptions.
- Scrapers call `self._fetch()` only — never raw `httpx` directly. All rate limiting, backoff, and

cookie reuse flow through `_fetch()`.

- Scrapers never write to DB. They return normalized objects. Task functions handle all persistence.
- All queries filter `WHERE deleted_at IS NULL`. Nightly ARQ cron hard-deletes rows past retention period.
- ARQ tasks update `chapter.scrape_status` on every chapter completion — not in a batch at the end. Enables partial reads mid-scrape.
- Cookie cache key: `"cookies:{source_key}"`. Value: JSON array of cookie dicts. TTL: `COOKIE_CACHE_TTL_SECS`.
- On 403 from source after cookie reuse: return HTTP 428 to iOS. Do not retry indefinitely server-side.
- All file paths in DB are relative to `BLOCK_VOLUME_PATH` — never absolute paths.
- `python-oracledb` thin mode ignores NLS environment variables. Set timezone explicitly in connection args if needed.

9.2 iOS

- MVVM strictly enforced. Views own no logic. ViewModels call services. Services call `APIClient` or `SwiftData` context.
- Cookie extraction fires automatically on every `webView(_:didFinish:)` — not only on Scrape tap. Cookies kept fresh in `CookieStore.swift` throughout the session.
- Minimum deployment target: iOS 17.2. All SwiftData predicate behaviour tested on 17.2 simulator specifically.
- All async calls use structured concurrency (`async/await` + `Task`). No completion handlers.
- Reading progress debounced: 5s minimum between API writes. Always written on view dismiss.
- HTTPS only. `API_BASE_URL` always starts with `https://`. No ATS exceptions in `Info.plist`.
- All design tokens (colors, spacing, type sizes) from `DesignSystem` package — no hardcoded hex values in feature modules.

9.3 Git

- Branch naming: `feature/{name}`, `fix/{description}`, `chore/{task}`
- Commit prefix: `[ios]` or `[backend]` followed by imperative description.
- `.env.oci` gitignored. Oracle wallet files gitignored. Never commit credentials or cert files.
- OpenAPI YAML files in `docs/api-contracts/` committed with every route change — before

merging.

---

10. Deferred & Open Items

Item	Status
Alembic migrations	Will be added before first real data is stored. Schema currently managed by `create_all()`.
Playwright stealth tuning	`playwright-stealth` patches will need per-site tuning as bot detection fingerprints evolve. Ongoing maintenance.
Author dedup v2	v1: exact string match. v2: `source_author_id` mapping table for cross-source canonical author records.
HTTPS on local dev	Local dev uses HTTP on localhost. Local `docker-compose.yml` variant skips Certbot and Nginx SSL. iOS Debug scheme points to HTTP localhost; Release scheme to HTTPS domain.
Download enforcement	5GB per-tab limit tracked in SwiftData. `DownloadService` calculates usage via `FileManager` and blocks downloads at limit.
Oracle Cloud account	Pending. Local dev uses PostgreSQL in Docker with SQLAlchemy. Switch to Oracle: change `DATABASE_URL`, add wallet, replace `postgresql` dialect with `oracle+oracledb`. Schema is dialect-agnostic.
Playwright scale	Max 2 browser instances on single VM. Cookie-harvest primary path minimises Playwright usage. Future: dedicate a VM split to a browser worker if scrape queue grows.
Thumbnail upload	User can null thumbnail. Custom user-uploaded thumbnails deferred to v2.
Fanfic tags	Not implemented in v1 per product decision. Architecture supports adding later without schema changes.